die Mobiliar

Vom Frontendmonolith zu Microfrontends

Emanuel Indermühle

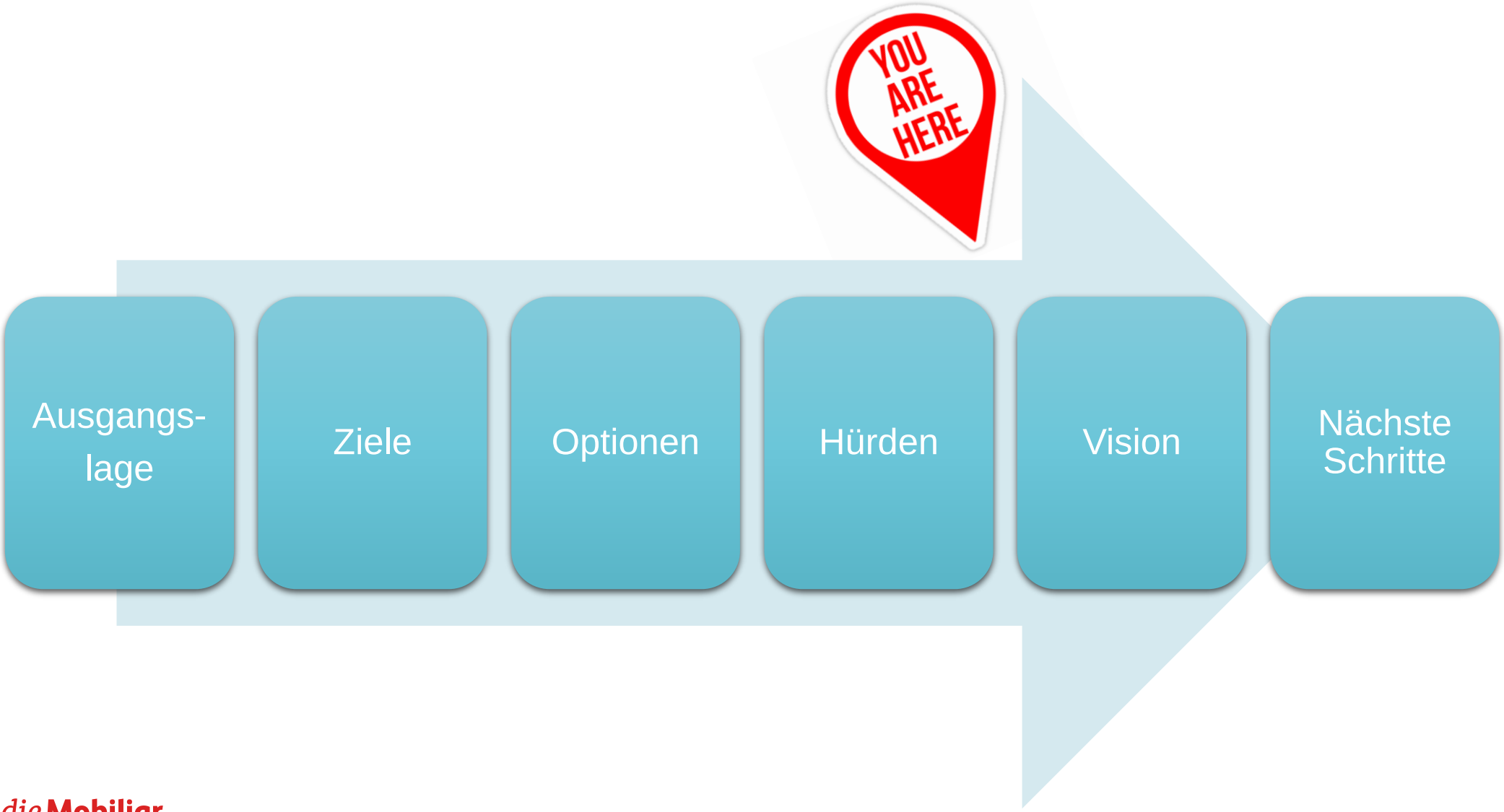
Emanuel Indermühle

PhD in Computer Science, Uni Bern

Technologie Architekt, die Mobiliar

Scrum-Rolle Product Owner der
Joint Application Platform, die Mobiliar

Auf unserem Weg vom Frontendmonolith zu Microfrontends



Ausgangslage

Wie alles begann

Ausgangslage – B2E Portal

Projekt

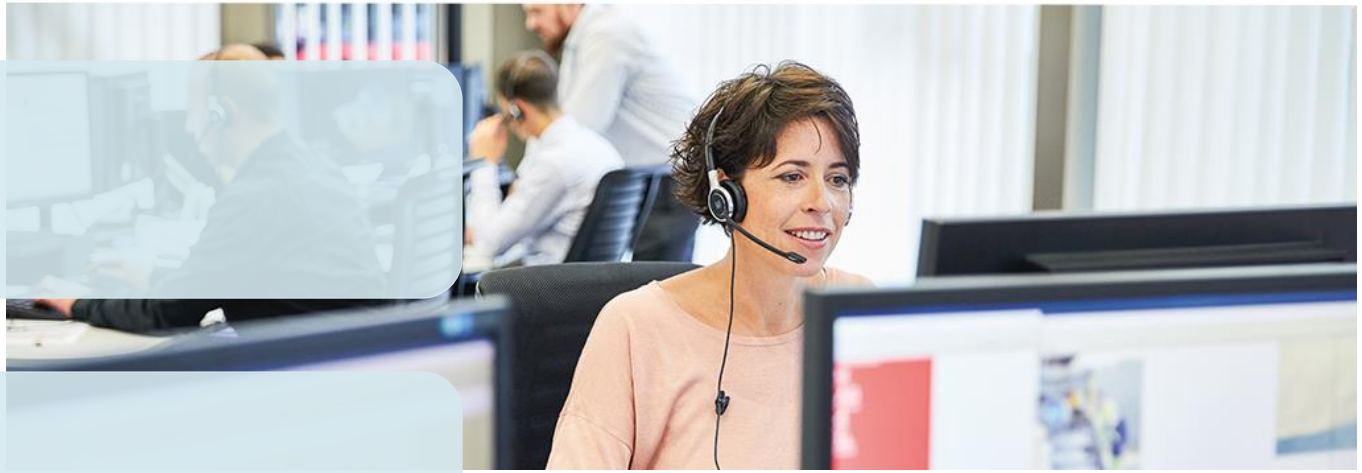
- B2E Portal
- Schadensplattform
- Angular JS
- Erfolgreiche Einführung Q1 2015

Organisation

- Ein Frontend Team
- Ein Architekt.

Architektur

- 2-Way Data Binding
- Monolith im Backend
- Release 2x jährlich
- Offline Anforderung



Ausgangslage – B2E Portal – Die Vertriebsplattform

Projekt

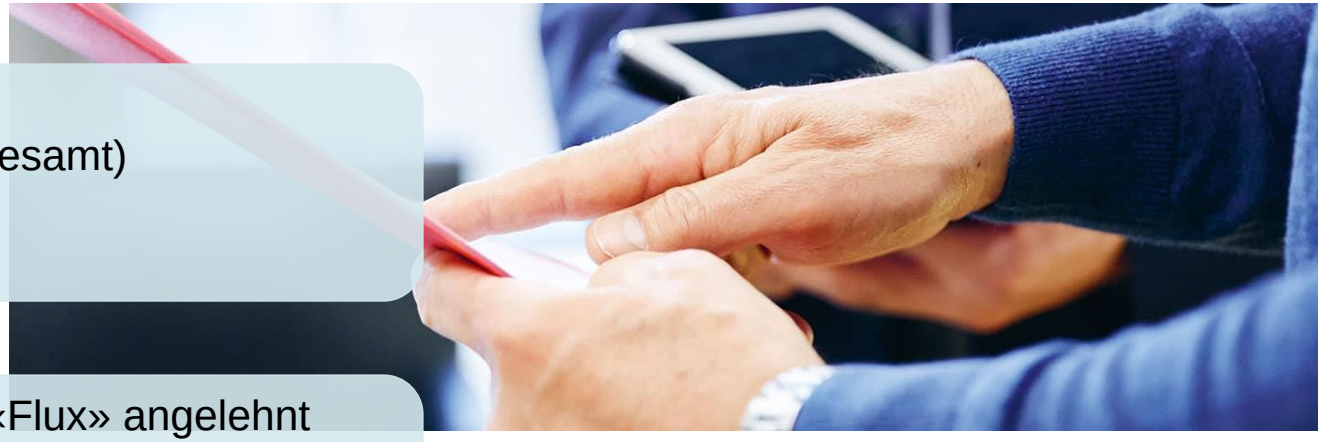
- Gleiche Code-Base
- Viel Code-Sharing

Organisation

- 3 neue Teams (5-7 insgesamt)
- 2 neue Architekten

Architektur

- State Management an «Flux» angelehnt
- Microservices im Backend
- Agiles Releasing
- Offline-Anforderung



Ausgangslage - Probleme

1 Code-Base

- Unterschiedliche Architekturen
- Komplexität des Builds
- Lokale Änderungen -> Globale Auswirkungen
- Fast 25 Mitarbeiter pushen Commits

1 SPA

- Viel Koordination für Releasing und Testing
- Performance Probleme (im IE)

Offline

- Kostet viel
- Keine Verifikation

LifeCycle

- Schwierig
- Bower zu NPM
- JavaScript zu Typescript

Ziele der Modularisierung

Weniger Koordination	• Keine Abstimmungsmeetings • Keine Feuerwehrübungen
Weniger Komplexität	• Entwickler gewinnen die Übersicht
Lokale Auswirkungen	• Changes in Vertriebsportal verursachen keinen Bugs im Schadenportal
Getrennte Releases	• Bugfix Releases muss nicht mit 25 Entwickler abgesprochen werden.
Lifecycle	• Teams können LifeCycle unabhängig angehen
Architektur	• Teams können App-Architektur selber wählen

Optionen

Optionen – «Code Base»

Monorepo

- Einfacheres Code Sharing
- Weniger Libs
- Einfacheres Testing
- Weniger Aufwand

Multirepo

- Weniger Code Sharing
- Kleinere SPAs einfacher Build
- Lokaler Change -> lokale Auswirkung

Optionen – «Navigation»

Monopage

- z.B. «Single SPA»
  - Performance Navigation
- z.B. «Scion Workbench»?

Multipage

- Mehrere SPAs
  - Technisch einfacher
  - Legacy-freundlich
  - Weniger Abhängigkeit
  - Ressourcen schonender

Optionen – «Code Sharing»

Libs

- Technisch einfach

iframe

- Unabhängiges Deployment
- Technisch einfach
- Eingeschränktes UX

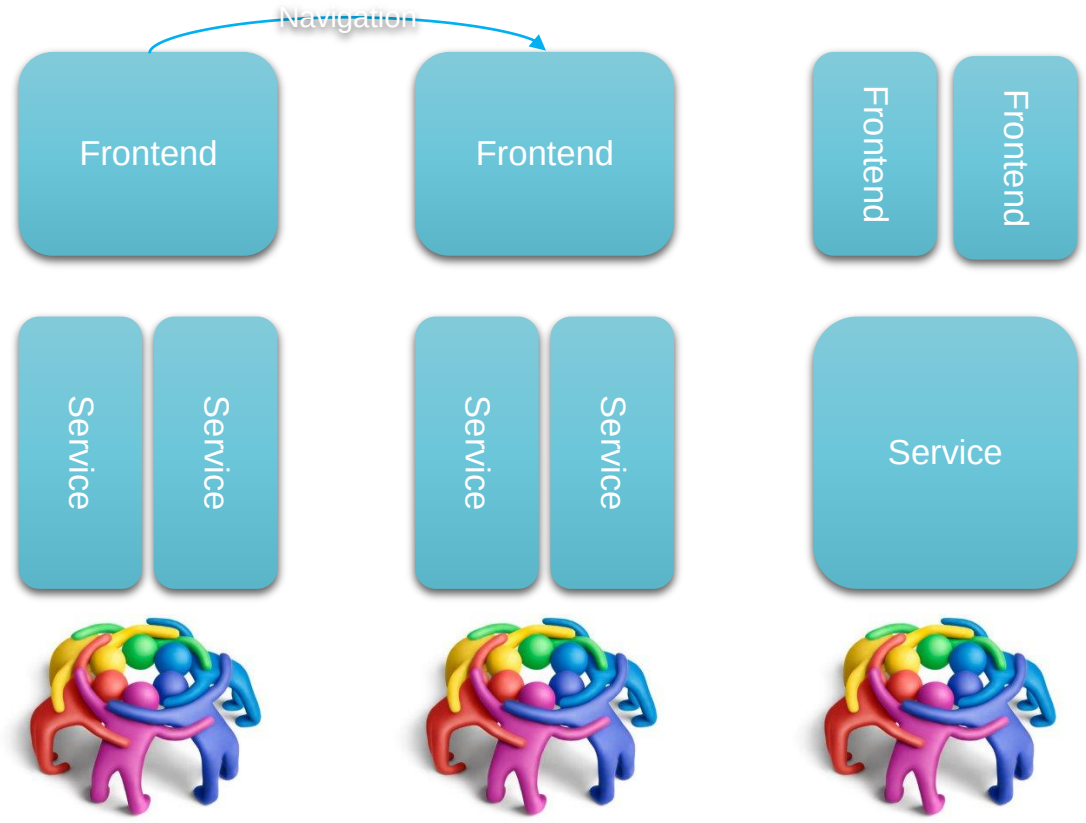
Web Components

- Unabhängiges Deployment
- Experimentell
- Schlechte Entwickler Ergonomie

Vision aus der Sicht «Feature Team»

Ein Feature Team kann…

- unabhängig Deployen
- unabhängig Lifecycle machen
- unabhängig das Framework wählen



Let’s Go

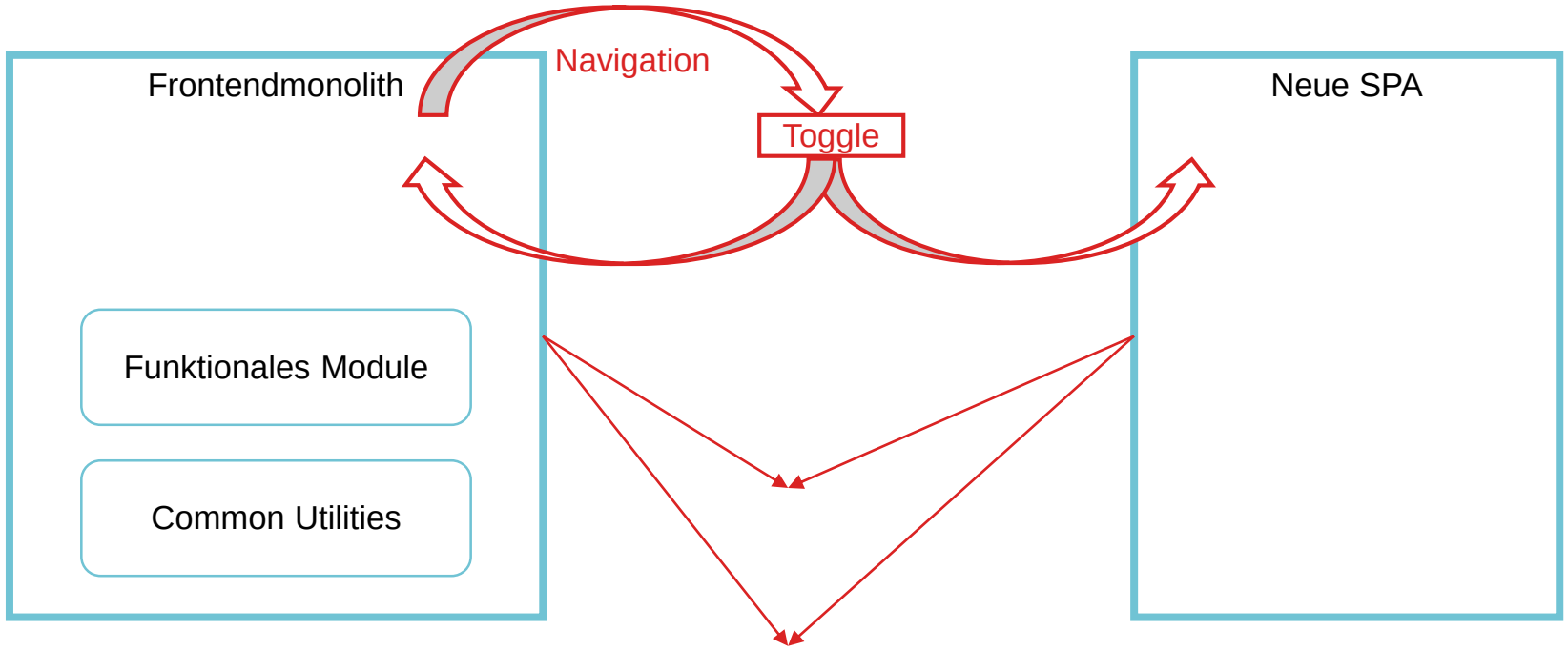
Multirepo

Multipage

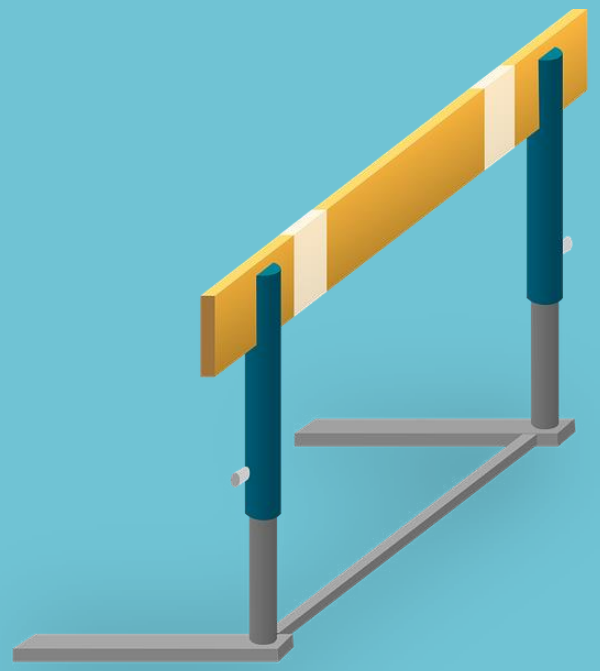
Libs

iframe

Modularisierung in Action

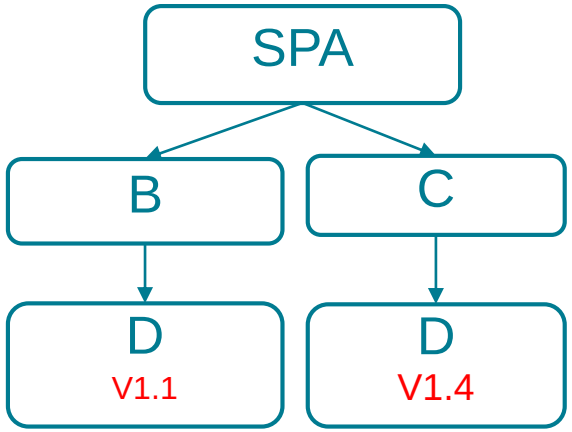


Hürde «Dependency-Hell»

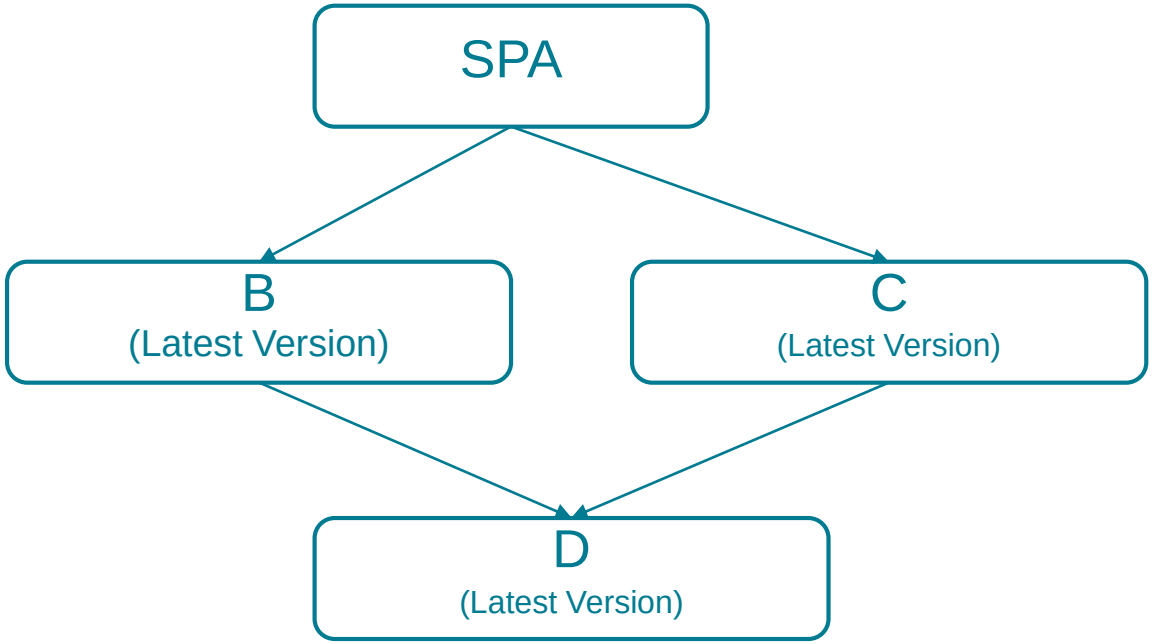


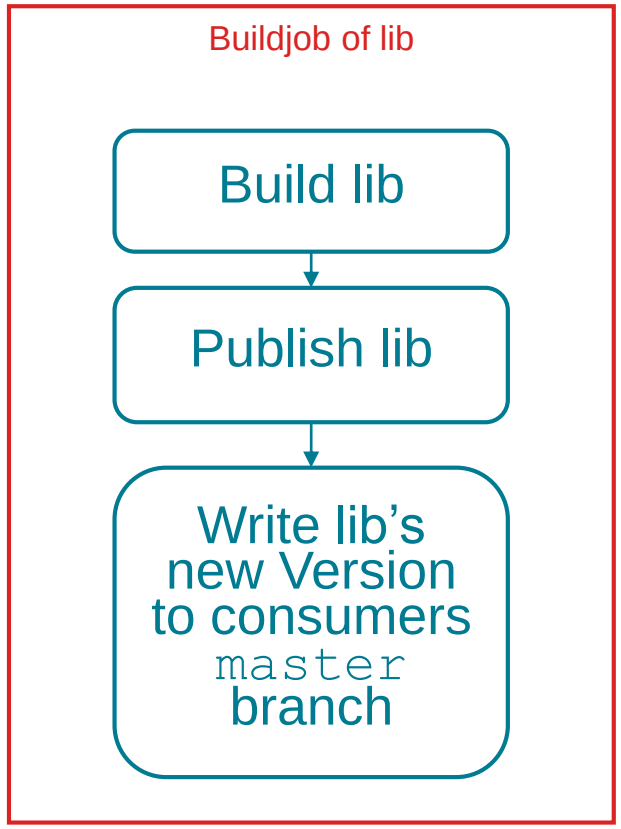
Hürde «Dependency-Hell»

- Welche Versionen sind miteinander kompatibel?

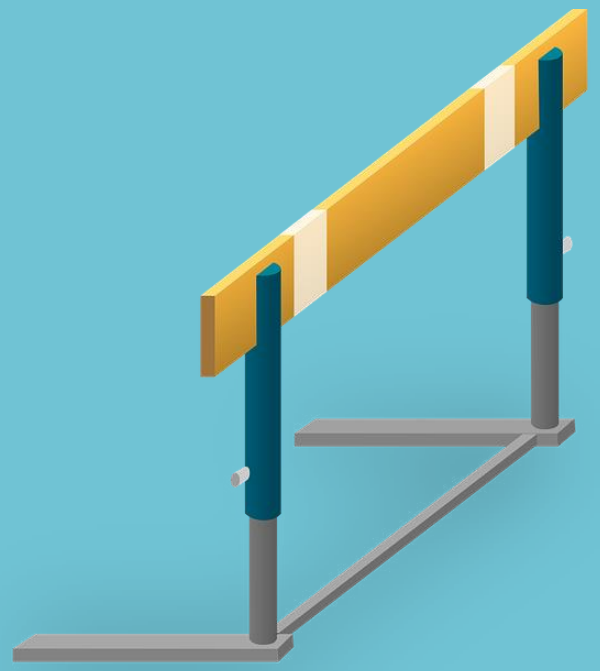


Hürde «Dependency-Hell» - Dependency Propagator

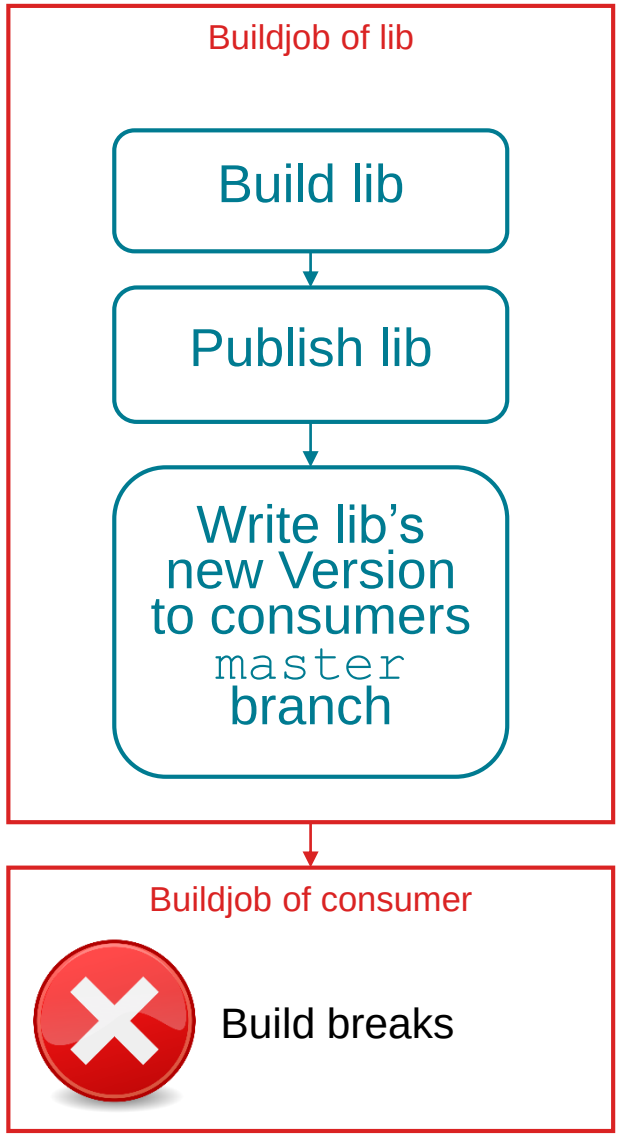




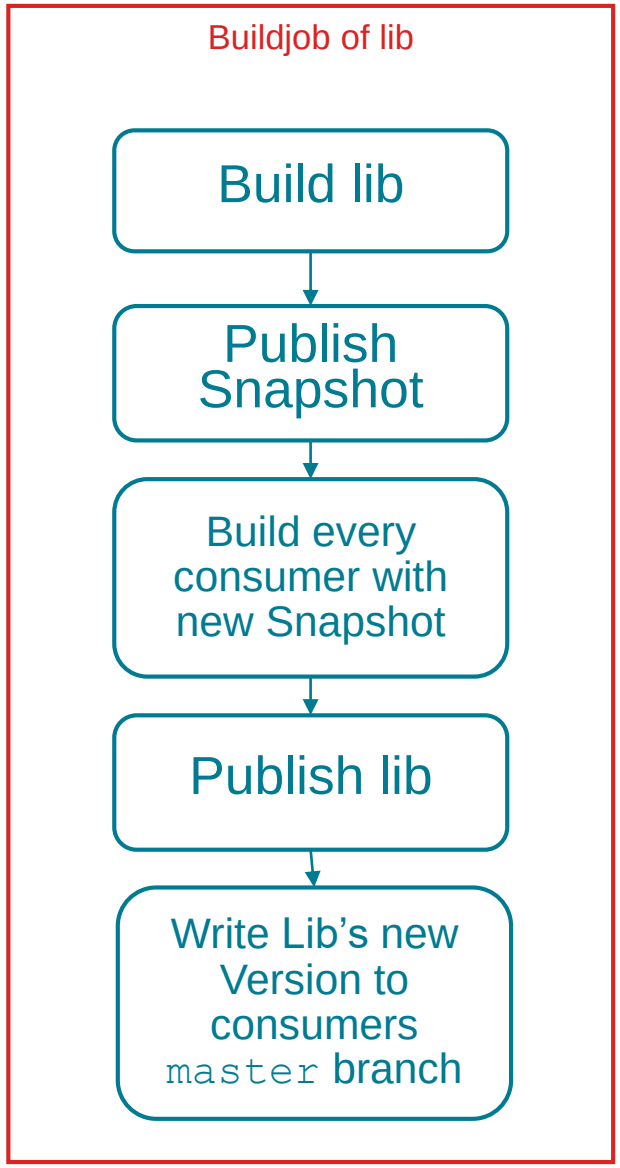
Hürde «Breaking Changes»



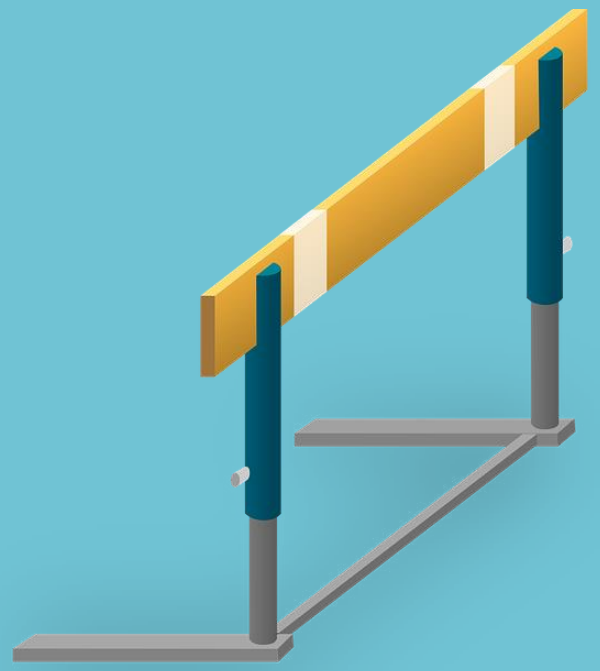
Hürde «Breaking Changes»



Hürde «Breaking Changes» – CDC Testing



Hürde «Angular Migration»



Hürde «Angular Migration»

Angular JS

Version: **1.5**, 1.6, 1.7

Angular

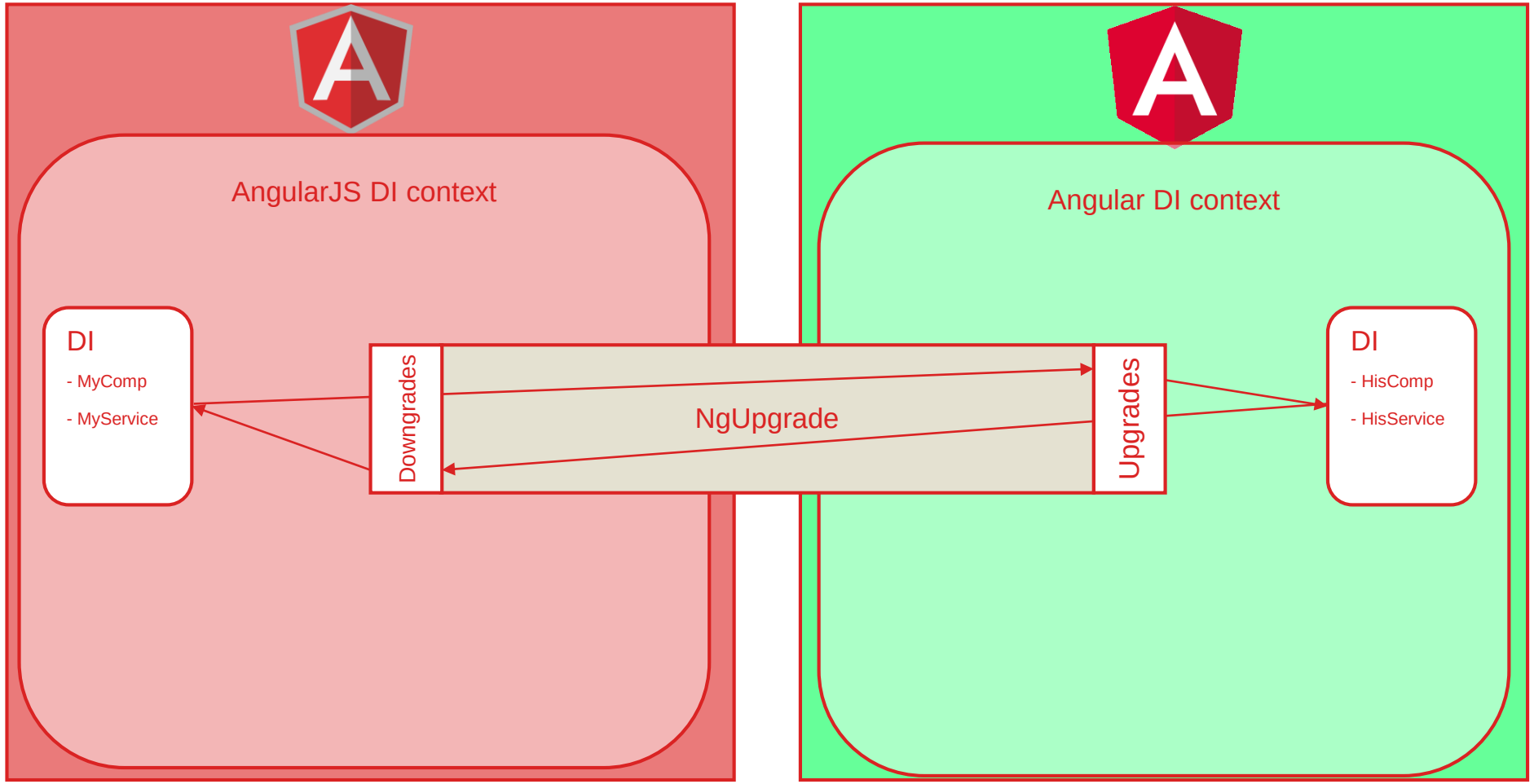
Version: 2, 4, **5**, 6, **7** (Semver)

Hürde «Angular Migration» - Eine neue Programmiersprache?

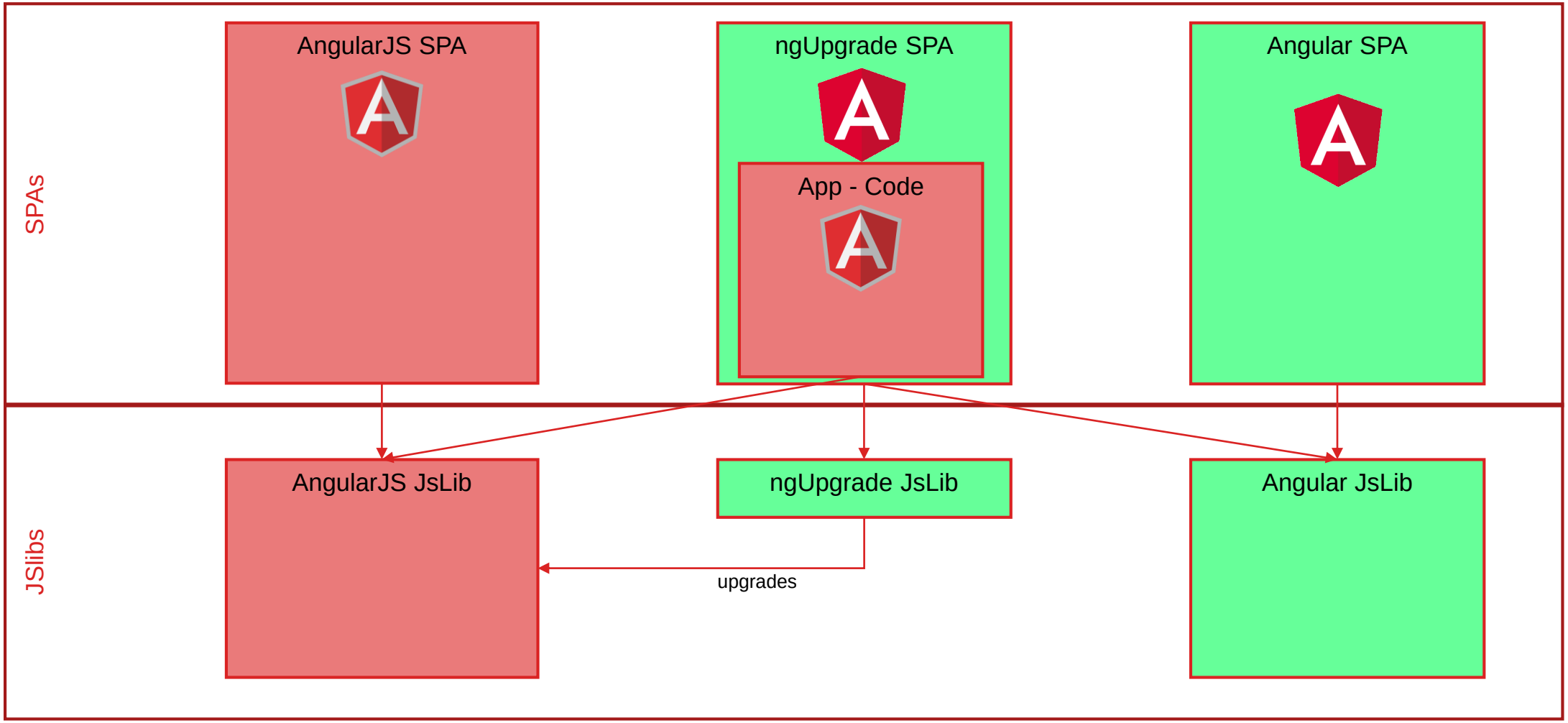
```
angular
    .module('b2e.ui.utils.disabled.directives', [])
    .directive('b2eDisabled', function b2eDisabledDirective() {
        return {
            restrict: 'EA',
            require: ['b2eDisabled', '?^^b2eDisabled'],
            controllerAs: 'b2eDisabledCtrl',
            controller: function B2eDisabledCtrl($scope, $element, $attrs) {
                var ctrl = this;
                var unregisterExpression;

                ctrl.$onInit = function() {
                    if (!ctrl.parentCtrl) {
                        ctrl.parentCtrl = parentDummyCtrl;
                    }
                    $scope.$on('$destroy',
                           ctrl.parentCtrl.registerChild(ctrl));

                    var expression = $attrs.b2eDisabled || $attrs.when;
                };
            }};
    });
```

```
import {Component, Inject, Input} from '@angular/core';
import {PORTAL_NAV_SERVICE} from '@mobi/b2e-utils-ng-jslib';

@Component({
    selector: 'vvn-vertrag-angebot-liste',
    templateUrl: './angebotsliste.component.html',
})
export class AngebotslisteComponent {
    @Input() geschaeftsnummer: string;
    @Input() partnerFachnummer: string;
    @Input() isLoading: boolean;

    constructor(@Inject(PORTAL_NAV_SERVICE) private pn: PortalNavigator) {
    }

    public neuesAngebotErstellen = (): void => {
        const baseUrl = '/vvn-vertrieb-b2e/vvn/produktauswahl/';
        return this.pn.navigateTo(baseUrl);
    }
}
```

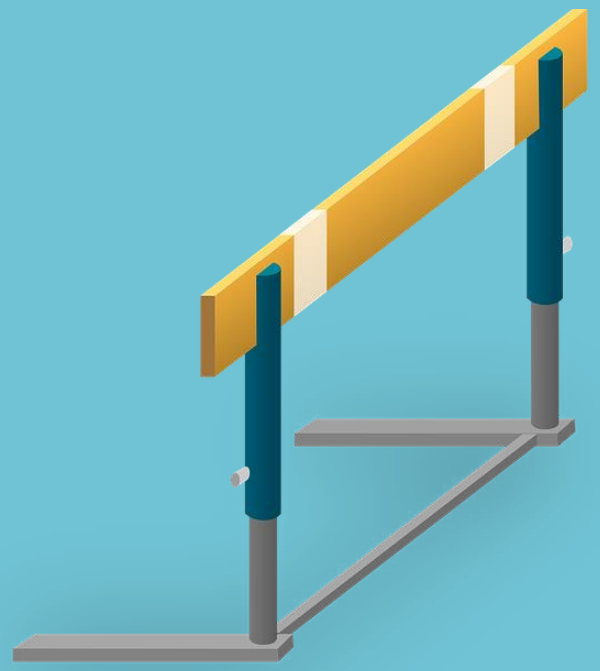
Hürde «Angular Migration» - ngUpgrade



Hürde «Angular Migration» - unterschiedlichen Upgrade Stufen

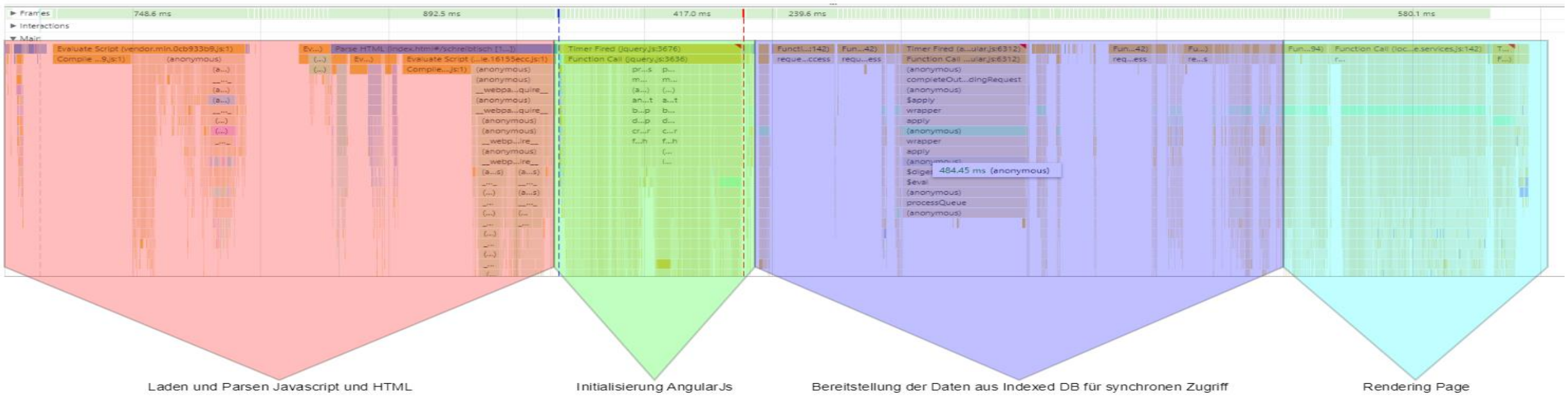


Hürde «Performance»



Hürde «Performance» - Loading

- Initiale Ladezeit der SPAs
- Navigation zwischen SPAs ist zu langsam



Hürde «Performance» - Loading

Laden

- Angular + Angularjs + ngUpgrade + Viel Code
- Modularisierung
- Angular Migration
  - AnuglarJs und ngUpgrade fallen weg, dafür haben wir Treeshaking

Initialisierung

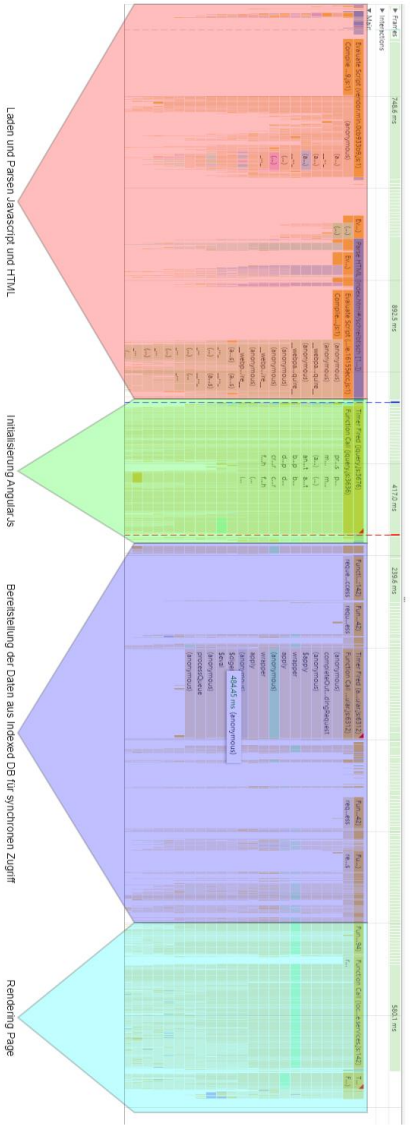
- Anuglar + AngularJS
- Angular Migration

Bereitstellung Stammdaten

- Offline Anforderung
- Lazy loading
  - Sync zu Async Rewrite (Aufwand Gross)

Rendering

- AngularJS
- Angular Migration



Hürde «Performance» - Dilemma

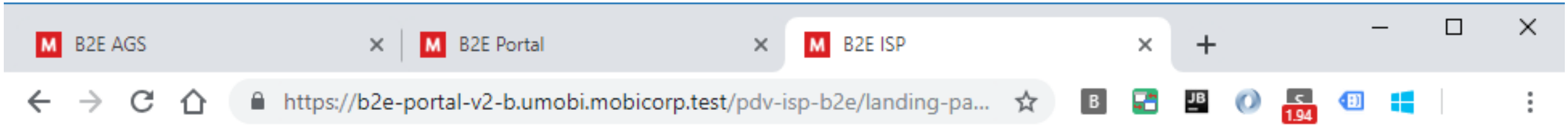
Modularisierung

- Angular Migration nötig
- Wegen Performance

Angular Migration

- Modularisierung nötig
- Wegen Individuellem LifeCycle

Hürde «Performance» - Tablösung



Wie?

- Chrome Extension
  - Navigation wird abgefangen
- Zuordnung von Root-Context zu Bowser-Tab

Hilfts?

- SPAs bleiben geladen,
- mehrfache Initialisierung entfällt.

Wie lange?

- Bis die SPAs schnell laden
  - Migration und Modularisierung entsprechend fortgeschritten sind.

Rückblick - Ausblick

Rückblick – Ziel erreicht?

- Koordination / Komplexität ✓
- Lokale Änderung -> Lokale Auswirkungen ✓
- Unabhängiges Releasing
  - Libs verhindern wirklich unabhängiges Releasing
- LifeCycle
  - Libs erzwingen orchestrierten LifeCycle
- Freiheit in der App Architektur
  - Angular ist gesetzt
- Schneidung
  - SPAs sind keine Micro Frontends

Ausblick – Web Components

- Mit Angular Elements
  Standardisierter Build-Prozess
- Unabhängiges Releasing
- Unabhängiger LifeCycle
- Die Micro Frontends

Nächste Schritte

Angular Migration zentraler Libs	• Erlaubt Angular-Only SPAs
Update Angular 7 auf Angular 8	• Wir bleiben dran
Experimente mit Angular Elements	• Siehe Vision
Observability im Frontend	• Wie gesund sind unsere Frontends

Das Team

Martin Fischer

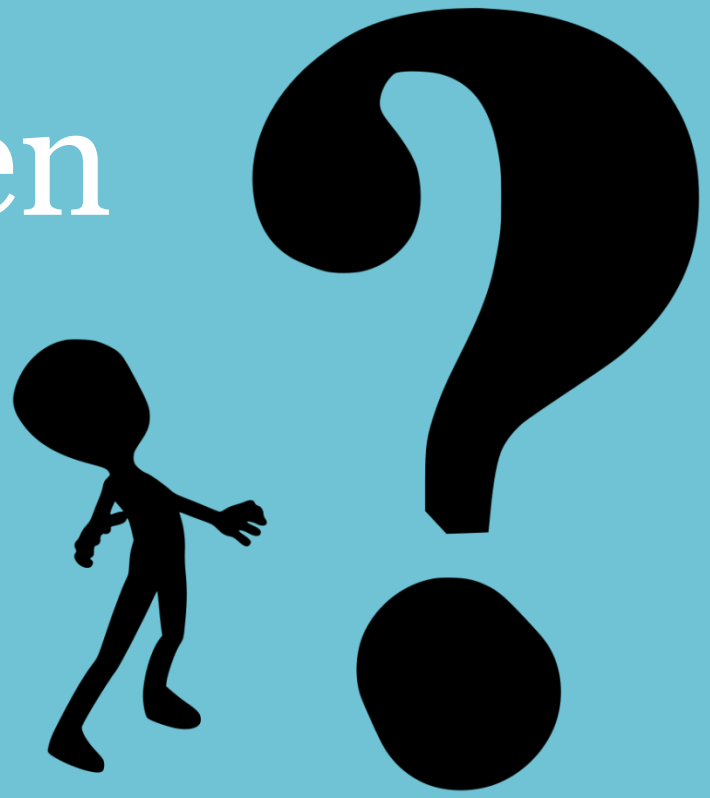
Tomas Trajan

Matthias Junker

Christian Kohler

Kevin Kreuzer

Christof Leuenberger

Roland Wyss

Haben wir Zeit für Fragen